F.CSM302 ҮЙЛДЛИЙН СИСТЕМ

БИЕ ДААЛТ 1

The Sleeping Teaching Assistant

Thread, Semaphore, Mutex ашигласан synchronization симуляци ба Tkinter дүрслэл

Тайлан

Хуваарилалтын код: **II-4-10**

Багийн гишүүд

№	Нэр	Оюутны код
1	А.Төгөлдөр	B242270001
2	Ц.Цогтжаргал	B241940005
3	Д.Отгон-Эрдэнэ	B222270083
4	О.Цэрэнлхавга	B232270113
5	Г.Түвшинсайхан	B242270085

GitHub: https://github.com/B242270001/II-4-10_SleepingTeachingAssistant

Улаанбаатар хот, 2026 он

Гарчиг

Хураангуй 4

1. Удиртгал 5

1.1 Сэдвийн холбогдол 5

1.2 Асуудлын тодорхойлолт 5

1.3 Зорилго, зорилт 5

1.4 Системийн архитектур ба хэрэгжүүлсэн шийдэл 5

1.5 Төслийн бүтэц ба хэрэгслүүд 7

10. Дүгнэлт 8

10.1 Хийгдсэн ажил 8

10.2 Туршилтын дүн 8

10.3 Илэрсэн хязгаарлалт 13

10.4 Цаашид сайжруулах чиглэл 14

10.5 Эцсийн дүгнэлт 14

Ашигласан материал 15

Зургийн жагсаалт

Зураг 1. Системийн архитектурын блок схем 6

Хүснэгтийн жагсаалт

Хүснэгт 1. Core-ийн synchronization объектууд 6

Хүснэгт 2. Төслийн бүтэц ба хэрэгслүүд 7

Хүснэгт 3. Туршилтын орчин 8

Хүснэгт 4. Туршилтын нэгдсэн дүн 8

Хүснэгт 5. Автомат тестүүдийн ангиллаар задаргаа 9

Хүснэгт 6. Багийн 11 автомат тестийн дэлгэрэнгүй 9

Хүснэгт 7. Edge case 27 тестийн дэлгэрэнгүй (tests/test_edge_cases.py), бүгд тэнцсэн 10

Хүснэгт 8. UI сценариудын төлөвлөсөн оролт ба хүлээгдэх үр дүн 11

Хүснэгт 9. Core дээр шууд (UI-гүй) ажиллуулсан бодит хэмжилт 12

Хүснэгт 10. UI дээр харагдсан тоо ба core-ийн бодит тооны харьцуулалт 12

Хүснэгт 11. UI сценариудын дэлгэц дээрх хэмжилт 13

Хураангуй

Энэхүү бие даалтаар Silberschatz-ийн *Operating System Concepts* (10th edition) сурах бичгийн 7-р бүлгийн “The Sleeping Teaching Assistant” програмчлалын төслийг Python хэл дээр хэрэгжүүлэв. Бодлогод нэг багшийн туслах (TA) болон хязгаарлагдмал тооны хүлээлгийн сандал, олон оюутан оролцоно. Оюутан байхгүй үед TA унтаж, оюутан ирэхэд сэрнэ. Сандал дүүрэн үед ирсэн оюутан буцна.

Төслийг хоёр хэсэгт хуваасан. **Core** (`ta_core.py`) нь TA болон оюутан бүрийг бодит thread болгон ажиллуулж, хүлээлгийн дарааллыг Lock (mutex)-ээр хамгаалж, унтах ба сэрэхийг Semaphore-оор хэрэгжүүлсэн. Core нь UI-аас хамааралгүй бөгөөд thread бүр өөрийн үйлдлийг event болгон дарааалалд (`queue.Queue`) хийдэг. **UI** (`ui.py`) нь Tkinter дээр бүтсэн dashboard бөгөөд тэр event-үүдийг уншиж TA, сандал, оюутныг хөдөлгөөнт дүрслэлээр, мөн тоолуур, график, event түүхээр бодит цагт харуулна. Хэрэглэгч сандал, оюутны тоо, ирэх ба үйлчилгээний хугацааг тохируулж, START, PAUSE, STEP, RESET удирдлагаар симуляцийг хянана.

Туршилтыг гурван түвшинд хийсэн: багийн 11 автомат тест, 27 нэмэлт edge case тест, 11 UI сценари. Бүх тест тэнцсэн. Synchronization-ы гол шинж болох FIFO дарааалал, сандлын тоо хэтрэхгүй байх, TA нэг дор нэг оюутанд үйлчлэх, deadlock-гүй байх нь бүх тохиргоонд, түүний дотор дээд ачаалалд (20 сандал, 50 оюутан) биелсэн. Илэрсэн гол сул тал нь ачаалал ихтэй үед UI-ийн дүрслэл core-оос хоцрох явдал юм.

Түлхүүр үг: synchronization, thread, semaphore, mutex, Sleeping Teaching Assistant, Python, Tkinter.

1. Удиртгал

1.1 Сэдвийн холбогдол

Орчин үеийн үйлдлийн систем олон процесс, thread-ийг зэрэг ажиллуулдаг. Эдгээр нь нийтийн нөөц (санах ой, дараалал, төхөөрөмж)-ийг хуваалцах үед синхрончлол алдагдвал race condition, deadlock зэрэг ноцтой алдаа гарна. Тиймээс mutex ба semaphore-оор нөөцийг зөв хамгаалж, thread-үүдийг зөв унтуулж сэрээх нь үйлдлийн системийн гол ойлголтуудын нэг юм. Sleeping Teaching Assistant бодлого нь эдгээр ойлголтыг өдөр тутмын нэг ойлгомжтой нөхцөлөөр харуулдаг сонгодог жишээ учраас сургалтад өргөн хэрэглэгддэг.

1.2 Асуудлын тодорхойлолт

Нөхцөл дараах байдалтай:

- ТА-ийн өрөөнд нэг сандал (ТА), коридорт N хүлээлгийн сандал байна.
- Оюутан байхгүй бол ТА унтана.
- Оюутан ирэхэд ТА унтаж байвал сэрээнэ. ТА завгүй бол сул сандалд суугаад хүлээнэ. Сул сандал байхгүй бол буцаж явна, хэсэг хугацааны дараа дахин ирнэ.
- Оюутнууд FIFO дарааллаар үйлчлүүлнэ.

Хэрэгжүүлэлтэд хоёр төрлийн алдаанаас сэргийлэх шаардлагатай. Нэгдүгээрт, хүлээлгийн дараалал болон ТА-ийн төлөвийг олон thread зэрэг өөрчлөхөөс (race condition). Хоёрдугаарт, ТА унтсан атлаа оюутан хүлээж үлдэх, эсвэл оюутан ТА-г сэрээж чадалгүй гацах (lost wakeup, deadlock) байдлаас.

1.3 Зорилго, зорилт

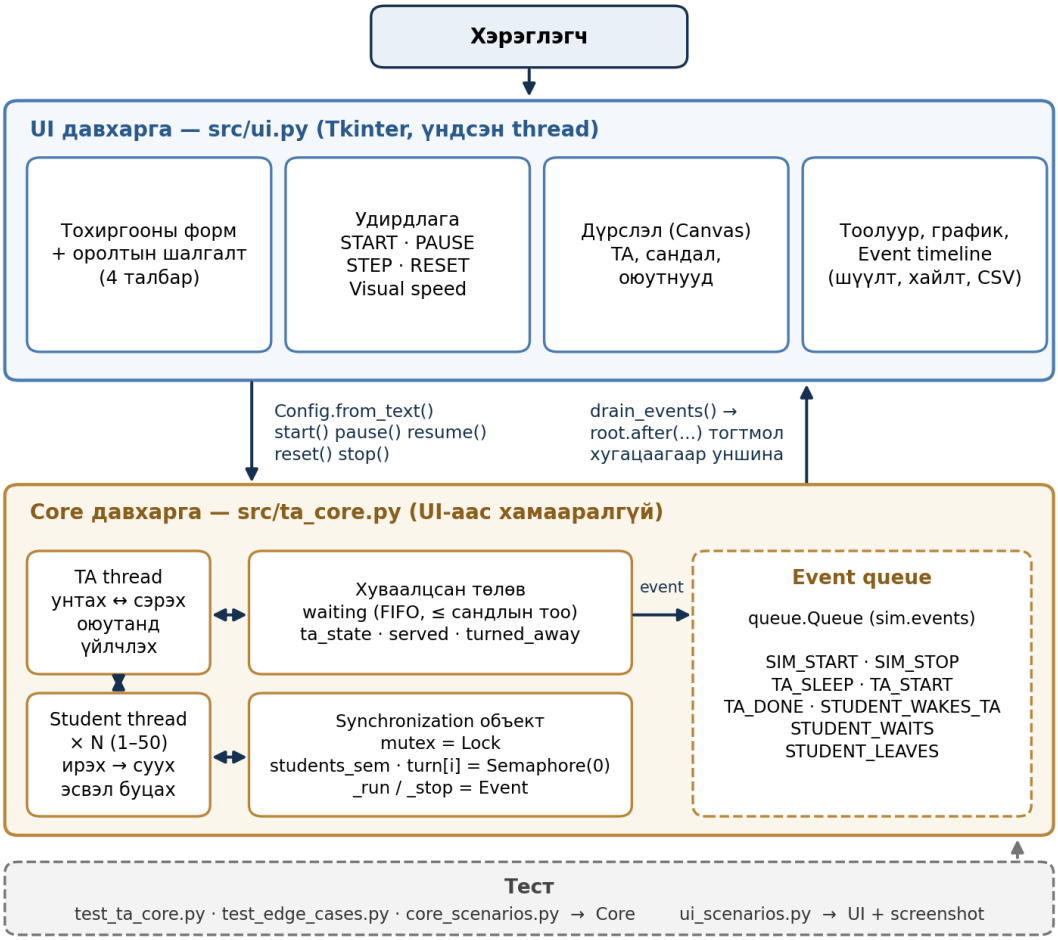
Зорилго: Sleeping Teaching Assistant бодлогыг thread, semaphore, mutex ашиглан зөв хэрэгжүүлж, ажиллагааг нь харагдахуйц дүрслэх.

Зорилтууд:

- UI-аас хамааралгүй, туршиж болохуйц core симуляци бичих (`ta_core.py`).
- Core-ийн бодит thread-үүдийн төлөвийг харуулсан интерактив Tkinter dashboard хийх (`ui.py`).
- Оролтыг шалгаж, буруу утгыг ойлгомжтой мессежээр мэдэгдэх.
- START, PAUSE, STEP, RESET удирдлага, event түүхийг шүүх, хайх, CSV болгон хадгалах боломж олгох.
- Автомат тест, edge case тест, UI сценариар зөв байдлыг баталгаажуулах.

1.4 Системийн архитектур ба хэрэгжүүлсэн шийдэл

Шийдэл нь номын сонгодог загвартай ижил. Систем нь хоёр давхарга: UI-аас хамааралгүй **Core** ба Tkinter **UI**. Thread бүр өөрийн үйлдлийг (оюутан ирсэн, суусан, буцсан, ТА сэрсэн, үйлчилж эхэлсэн, дууссан, унтсан) event болгон `queue.Queue`-д хийнэ. Tkinter-ийн үндсэн thread нь `root.after(...)`-ээр тэр дарааллыг уншиж дэлгэц дээр зурдаг. Иймээс дүрслэл нь core-ийн бодит ажиллагааг харуулж, UI thread нь core-ийн хугацаанд нөлөөлөхгүй.



Зураг 1. Системийн архитектурын блок схем

Хүснэгт 1. Core-ийн synchronization объектууд

Объект	Төрөл	Үүрэг
`mutex`	`threading.Lock`	Хүлээлгийн дараалал, TA-ийн төлөв, тоолуурыг хамгаална
`students_sem`	`Semaphore(0)`	Хүлээж буй оюутны тоо. TA үүн дээр *wait* хийж унтана, оюутан *signal* өгч сэрээнэ
`turn[i]`	`Semaphore(0)`	i-р оюутны хувийн semaphore. TA үйлчилж дуустал оюутан үүн дээр хүлээнэ
`_run`	`threading.Event`	Set = ажиллаж байна, clear = түр зогссон (pause)
`_stop`	`threading.Event`	Бүх thread-ийг зогсоох дохио

1.5 Төслийн бүтэц ба хэрэгслүүд

Хүснэгт 2. Төслийн бүтэц ба хэрэгслүүд

Зүйл	Агуулга
`src/ta_core.py`	Core: *Config*, *Simulation*, thread-үүд, консол горим
`src/ui.py`	Tkinter dashboard
`src/test_ta_core.py`	Багийн 11 автомат тест (*unittest*)
`tests/`	27 edge case тест, core ба UI сценарийн скрипт, screenshot
`docs/`	Report, Manual
Хэл, сан	Python 3.14, Tk 9.0. Стандарт сан: *threading*, *queue*, *tkinter*, *unittest*
Хувилбарын удирдлага	Git, GitHub (https://github.com/B242270001/II-4-10_SleepingTeachingAssistant)

10. Дүгнэлт

10.1 Хийгдсэн ажил

Sleeping Teaching Assistant бодлогыг бодит thread, mutex, semaphore ашиглан хэрэгжүүлж, UI-аас хамааралгүй core болон Tkinter dashboard-ыг бүтээсэн. Dashboard нь оролтыг шалгаж, симуляцийг START, PAUSE, STEP, RESET-ээр удирдах, TA ба оюутнуудын хөдөлгөөнийг, тоолуур ба графикийг бодит цагт харуулах, event түүхийг шүүж, хайж, CSV болгон хадгалах боломжтой. Хэрэглэгчийн болон operational гарын авлага (Manual) бэлтгэсэн.

10.2 Туршилтын дүн

Туршилтыг гурван түвшинд хийсэн. Доорх хүснэгтүүдэд туршилтын орчин, нэгдсэн дүн, тест болон сценари тус бүрийн хэмжигдэхүүнийг өгөв. Симуляци санамсаргүй тул тоо ажиллуулалт бүрт ялгаатай.

Хүснэгт 3. Туршилтын орчин

Зүйл	Утга
Үйлдлийн систем	macOS 26.6.2
Python / Tk	3.14.10 / Tk 9.0
Төслийн хувилбар	main, commit 198e554 (release-ийн өмнө эцсийн commit/tag-аар шинэчилнэ)
Core	src/ta_core.py (thread, Semaphore, Lock)
UI	src/ui.py (Tkinter dashboard), Visual speed 1.00×
Огноо	2026.10.28

Хүснэгт 4. Туршилтын нэгдсэн дүн

Түвшин	Файл	Тестийн тоо	Үр дүн	Хугацаа
Багийн автомат тест	`src/test_ta_core.py`	11	11 / 11 тэнцсэн	14.97 сек
Edge case тест	`tests/test_edge_cases.py`	27	27 / 27 тэнцсэн (*OK*)	22.8 сек
Core сценари	`tests/core_scenarios.py`	6 сценари (S1–S6)	Бодит тоо JSON-д хадгалагдсан	~2 мин
UI сценари	`tests/ui_scenarios.py`	11 сценари (S1–S11)	11 / 11 тэнцсэн	~3.5 мин
Нийт		**38 автомат тест + 11 UI сценари**	**Бүгд тэнцсэн**	

Хүснэгт 5. Автомат тестүүдийн ангиллаар задаргаа

Файл	Тестийн анги	Тест	Юуг шалгадаг
`test_ta_core.py`	TestValidation	2	Буруу оролт (8 хослол), зөв ба хязгаарын утга
`test_ta_core.py`	TestCorrectness	6	Invariant (сандал, давхцал, нэг дор нэг үйлчилгээ), ачаалал, 0 сандал, унтах ба сэрэх, deadlock
`test_ta_core.py`	TestControl	3	Pause / resume / stop, reset, давхар start
`test_edge_cases.py`	TestConfigBoundaries	9	Хязгаарын утга, nan/inf, хоосон зай, бүхэл тоо, seed, алдааны мессеж
`test_edge_cases.py`	TestLifecycleEdgeCases	7	Stop, pause, reset-ийн ер бусын дараалал, thread гоожилт
`test_edge_cases.py`	TestSimulationSemantics	11	FIFO, тоолуур, цаг хугацаа, TA-ийн төлөв, дээд ачаалал
Нийт		**38**	

Хүснэгт 6. Багийн 11 автомат тестийн дэлгэрэнгүй

№	Тест	Оролт / нөхцөл	Хүлээгдэх үр дүн
1	test_bad_inputs	8 буруу хослол (-1, 21, "abc", 0 …)	Бүгд ValueError
2	test_good_and_edge_inputs	3/5/1.5/2; 0/1/0.01/0.01; 20/50/60/60	Алдаагүй уншина
3	test_book_default	3 сандал, 5 оюутан, 2 сек	Invariant биелнэ, үйлчилгээ > 0, thread 0
4	test_overload_students_get_turned_away	1 сандал, 20 оюутан	Буцсан > 0, STUDENT_LEAVES гарна
5	test_zero_chairs	0 сандал, 5 оюутан	Үйлчлүүлсэн 0, буцсан > 0, хэн ч суухгүй
6	test_ta_sleeps_when_nobody_comes	Ирэх хугацаа 30 сек	TA унтсан, TA_START гарахгүй
7	test_student_wakes_ta	1 оюутан	STUDENT_WAKES_TA гарна
8	test_no_deadlock_many_configs	1/3, 2/10, 5/30, 3/3	Үйлчилгээ > 3, thread 0, invariant биелнэ
9	test_pause_resume_stop	pause → resume → pause → stop	Pause үед event 0, resume дараа үргэлжилнэ, гацахгүй
10	test_reset_with_new_config	reset(шинэ Config)	Тоолуур, event 0, шинэ тохиргоогоор ажиллана
11	test_double_start_rejected	start() хоёр удаа	RuntimeError

Хүснэгт 7. Edge case 27 тестийн дэлгэрэнгүй (tests/test_edge_cases.py), бүгд тэнцсэн

№	Бүлэг	Edge case	Оролт / нөхцөл	Хүлээгдэх үр дүн
1	Хязгаар	Сандлын хязгаар	0, 20 / 21, -1	0, 20 зөв. 21, -1 алдаа
2	Хязгаар	Оюутны хязгаар	1, 50 / 0, 51	1, 50 зөв. 0, 51 алдаа
3	Хязгаар	Хугацааны хязгаар	0.001, 60 / 0, 60.01	0.001, 60 зөв. 0, 60.01 алдаа
4	Хязгаар	nan, inf, -inf	Хугацааны талбар	ValueError
5	Хязгаар	Хоосон зай, 1e1	" 3 ", "1e1"	Зөв уншина (3, 10.0)
6	Хязгаар	Бүхэл тооны талбарт бутархай, текст, хоосон	"3.0", "5.5", "", "три"	ValueError
7	Хязгаар	Seed талбар	"", "-5" / "abc", "1.5"	None, -5 зөв. Бусад алдаа
8	Хязгаар	Алдааны мессеж	"abc"	"Сандлын тоо", "abc" орсон ойлгомжтой мессеж
9	Хязгаар	Simulation-д шууд буруу Config	num_chairs=-1	ValueError
10	Lifecycle	Эхлэхээс өмнө stop	stop()	Алдаа гарахгүй
11	Lifecycle	Хоёр удаа stop	stop(); stop()	Алдаа гарахгүй, thread 0
12	Lifecycle	Эхлэхээс өмнө pause	pause(); start()	Resume хүртэл зөвхөн SIM_START гарна
13	Lifecycle	Pause үед reset	pause(); reset()	Pause арилна, event цэвэр
14	Lifecycle	Буруу тохиргоотой reset	reset(Config(99))	ValueError, симуляци зогссон. Дараа нь хэвийн reset болно
15	Lifecycle	6 удаа дараалан reset	start → reset × 6	Thread гоожихгүй
16	Lifecycle	Event-ийн төгсгөл	stop	SIM_STOP яг нэг удаа, хамгийн сүүлд
17	Синхончлол	FIFO дараалал	5 сандал, 8 оюутан	Эхэлж суусан нь эхэлж үйлчлүүлнэ
18	Синхончлол	Ирэлт бүр хаана очсон	2 сандал, 10 оюутан	Бүгд суусан эсвэл буцсан. LEAVES тоо = turned_away
19	Синхончлол	Үйлчлүүлсэн ≤ суусан	3 сандал, 6 оюутан	served суусан тооноос хэтрэхгүй
20	Синхончлол	Цаг хугацаа	3 сандал, 10 оюутан	Event-ийн t буурахгүй
21	Синхончлол	current талбар	3 сандал, 5 оюутан	TA_START-д current = оюутан, TA_DONE-д None
22	Синхончлол	Эхний оюутан TA-г сэрээнэ	1 оюутан	SIM_START → WAKES_TA → WAITS → TA_START

№	Бүлэг	Edge case	Оролт / нөхцөл	Хүлээгдэх үр дүн
23	Синхрончлол	Дараалал хоосорвол TA унтана	1 оюутан, удаан ирнэ	TA_DONE-ий дараа TA_SLEEP
24	Синхрончлол	1 сандал, 1 оюутан	1 / 1	3-аас олон үйлчлүүлнэ, хэзээ ч буцахгүй
25	Синхрончлол	Сандал хангалттай	20 сандал, 10 оюутан	Хэн ч буцахгүй
26	Синхрончлол	Дээд ачаалал	20 сандал, 50 оюутан	Дараалал ≤ 20, давхцахгүй, thread 0, deadlock-гүй
27	Синхрончлол	0 сандал	0 сандал, 5 оюутан	TA унтсан хэвээр, хэн ч үйлчлүүлэхгүй

Хүснэгт 8. UI сценариудын төлөвлөсөн оролт ба хүлээгдэх үр дүн

ID	Нэр	Төрөл	Оролт	Хүлээгдэх үр дүн
S1	Үндсэн ажиллагаа	Хэвийн	3 / 5 / 1.0 / 1.5	TA эхэнд унтсан. Оюутан ирэхэд сэрж, дарааллаар үйлчилнэ
S2	Сандал дүүрэн	Хязгаар	1 / 10 / 0.3 / 1.5	Оюутнууд буцна. Дараалал сандлын тооноос хэтрэхгүй
S3	Сандал хангалттай	Хэвийн	10 / 3 / 1.5 / 0.5	Хэн ч буцахгүй, буцсан тоо 0
S4	0 сандал	Хязгаар	0 / 5 / 0.5 / 1.0	Хэн ч үйлчлүүлэхгүй, TA унтсан хэвээр
S5	Нэг оюутан	Хязгаар	1 / 1 / 2.0 / 0.5	Оюутан бүрт TA сэрж, дараа нь дахин унтана
S6	Дээд ачаалал	Хязгаар	20 / 50 / 0.2 / 0.5	Гацахгүй. Дараалал 20-оос хэтрэхгүй
S7	PAUSE / RESUME	Удирдлага	3 / 5 / 1.0 / 1.5	PAUSE үед хөдөлгөөн зогсоно, START дарахад үргэлжилнэ
S8	STEP	Удирдлага	3 / 5 / 1.0 / 1.5	STEP нэг дарахад нэг event гарна
S9	RESET	Удирдлага	3 / 5 / 1.0 / 1.5	READY төлөв, бүх тоолуур, event тоо 0
S10	Хязгаараас хэтэрсэн	Алдаа	25 / 5 / 1.0 / 1.5	Алдааны мессеж, симуляци эхлэхгүй
S11	Буруу оролт (текст)	Алдаа	abc / 5 / 1.0 / 1.5	Алдааны мессеж, симуляци эхлэхгүй

Хүснэгт 9. Core дээр шууд (UI-гүй) ажиллуулсан бодит хэмжилт

ID	Оролт	Хугацаа (сек)	Үйлчлүүлсэн	Буцсан	Сууса н	ТА сэрсэн	ТА унтсан	Дээд дараалал
S1	3 / 5 / 1.0 / 1.5	20	11	11	15	1	0	3
S2	1 / 10 / 0.3 / 1.5	20	11	434	13	1	0	1
S3	10 / 3 / 1.5 / 0.5	20	20	0	22	11	10	2
S4	0 / 5 / 0.5 / 1.0	15	0	120	0	0	0	0
S5	1 / 1 / 2.0 / 0.5	20	6	0	6	6	6	1
S6	20 / 50 / 0.2 / 0.5	20	33	2451	54	1	0	20

Хүснэгт 10. UI дээр харагдсан тоо ба core-ийн бодит тооны харьцуулалт

ID	Core: үйлчлүүлсэн	UI: үйлчлүүлсэн	Core: буцсан	UI: буцсан	UI-ийн үйлчлүүлсэн / Core
S1	11	9	11	12	82%
S2	11	1	434	78	9%
S3	20	21	0	0	—
S4	0	0	120	64	—
S5	6	6	0	0	100%
S6	33	1	2451	59	3%

Хүснэгт 11. UI сценариудын дэлгэц дээрх хэмжилт

ID	Хэмжсэн хугацаа / үйлдэл	Дэлгэц дээр харагдсан үр дүн	Үр дүн
S1	Эхлэхээс өмнө; 20 сек	Эхлэхээс өмнө ТА унтсан. 20 сек-ийн дараа үйлчлүүлсэн 9, буцсан 12, дараалалд 3, ТА HELPING	Тэнцсэн
S2	20 сек	Буцсан 78, дараалалд 1 (1 сандал дүүрэн), үйлчлүүлсэн 1 (UI хоцролт)	Тэнцсэн
S3	20 сек	Үйлчлүүлсэн 21, буцсан 0	Тэнцсэн
S4	15 сек	Үйлчлүүлсэн 0, буцсан 64, ТА SLEEPING, 0 / 0 seats	Тэнцсэн
S5	20 сек	1 оюутан үйлчлүүлсний дараа ТА дахин унтсан. Үйлчлүүлсэн 6, буцсан 0	Тэнцсэн
S6	20 сек	RUNNING, гацаагүй. Дараалал 20 / 20, буцсан 59, үйлчлүүлсэн 1 (UI хоцролт)	Тэнцсэн
S7	PAUSE дараа 4 сек	PAUSED үед event тоо 15 → 15 (өөрчлөгдөөгүй). START-ийн дараа 30 хүртэл үргэлжилсэн	Тэнцсэн
S8	PAUSE, STEP × 5	Event тоо 6 → 7 → 8 → 9 → 10 → 10. Сүүлийн STEP-д "WAITING FOR EVENT"	Тэнцсэн
S9	10 сек, RESET	RESET-ийн өмнө: үйлчлүүлсэн 5, буцсан 5, event 27. RESET-ийн дараа: READY, event 0	Тэнцсэн
S10	START	INVALID INPUT. "Сандлын тоо 0-20 хооронд бүхэл тоо байна."	Тэнцсэн
S11	START	INVALID INPUT. "'Сандлын тоо' нь бүхэл тоо байх ёстой (оруулсан: 'abc')."	Тэнцсэн

Synchronization-ы гол шинжүүд бүх тохиргоонд биелсэн:

- Хүлээлгийн дараалал mutex-ээр хамгаалагдсан тул сандлын тооноос хэтэрсэн, давхцсан бичлэг гараагүй (дээд ачаалал 20 сандал, 50 оюутанд дараалал хамгийн ихдээ 20).
- ТА нэг дор зөвхөн нэг оюутанд үйлчилсэн.
- FIFO дараалал баталгаажсан: эхэлж суусан оюутан эхэлж үйлчлүүлсэн.
- Semaphore-оор унтах ба сэрэх зөв ажиллаж, ТА оюутангүй үед унтаж, анхны ирсэн оюутан сэрээсэн.
- Deadlock болон thread гоожилт гараагүй (олон дахин reset хийсний дараа ч thread үлдээгүй).
- Сандал 0 үед хэн ч үйлчлүүлээгүй, сандал хангалттай үед хэн ч буцаагүй.

10.3 Илэрсэн хязгаарлалт

Туршилтаар дараах хязгаарлалтууд илэрсэн. Эдгээр нь core-ийн synchronization-ы зөв байдалд нөлөөлөөгүй.

- **UI-ийн дүрслэл ачаалалтай үед хоцрох.** Олон оюутан буцдаг тохиргоонд UI нь event-үүдийг тогтмол хурдаар тоглуулдаг тул дэлгэц дээрх “үйлчлүүлсэн” тоо core-оос бага харагдана (S2: core 11, UI 1; S6: core 33, UI 1). Core-ийн тоо зөв.

- **STEP товч.** Event буфер хоосон үед core-г үргэлжлүүлдэг тул дараагийн STEP-д нэгээс олон event нэг дор орж ирэх тохиолдол ажиглагдсан.
- **reset(new_config).** Буруу тохиргоотой дуудахад эхлээд stop() хийгдэж, дараа нь тохиргоо шалгагддаг тул симуляци зогссон хэвээр үлдэнэ. UI тохиргоог урьдчилан шалгадаг тул хэрэглэгчид тохиолдохгүй.
- **Seed.** UI-д seed талбар байхгүй тул сценарийг яг ижил тоогоор давтах боломжгүй. Seed нь консол горим ба тестэд ажиллана.
- **Орчин.** Туршилтыг macOS дээр хийсэн. UI сценарийн скрипт зөвхөн macOS-д ажиллана.

10.4 Цаашид сайжруулах чиглэл

- Event буфер хуримтлагдсан үед нэг frame-д олон event боловсруулах эсвэл дүрслэх хурдыг автоматаар нэмж, UI-г core-той нийцүүлэх.
- `reset`-д эхлээд тохиргоог шалгаад дараа нь `stop()` хийх.
- UI-д seed талбар нэмж, туршилтыг яг давтах боломжтой болгох.
- Тест файлуудыг нэг хавтсанд цэгцэлж, нэг командаар ажиллуулдаг болгох.
- Өөр үйлдлийн системд (Windows, Linux) тест, UI-г шалгах.

10.5 Эцсийн дүгнэлт

Төслөөр thread, mutex, semaphore-оор нийтийн нөөцийг хамгаалах, thread-үүдийг унтуулж сэрээх зарчмыг бодитоор хэрэгжүүлж, үр дүнг нь харагдахуйц дүрсэлж, олон түвшний тестээр баталгаажууллаа. Core-ийн synchronization бүх тохиргоонд зөв ажилласан бөгөөд илэрсэн хязгаарлалтууд нь голчлон дүрслэлийн хэсэгт байна. Эдгээрийг дээрх чиглэлээр засч сайжруулах боломжтой.

Ашигласан материал

[1] Silberschatz, A., Galvin, P. B., Gagne, G. *Operating System Concepts*. 10th Edition. Wiley, 2018. Chapter 7, Programming Project 10: The Sleeping Teaching Assistant.

[2] Python Software Foundation. *threading — Thread-based parallelism*. Python 3 documentation. https://docs.python.org/3/library/threading.html

[3] Python Software Foundation. *queue — A synchronized queue class*. Python 3 documentation. https://docs.python.org/3/library/queue.html

[4] Python Software Foundation. *tkinter — Python interface to Tcl/Tk*. Python 3 documentation. https://docs.python.org/3/library/tkinter.html

[5] Python Software Foundation. *unittest — Unit testing framework*. Python 3 documentation. https://docs.python.org/3/library/unittest.html

[6] Багийн GitHub repository: https://github.com/B242270001/II-4-10_SleepingTeachingAssistant

[7] II-4-10 баг. *Туршилт, Үр дүн* (testing_results.pdf). Туршилтын тайлан, 2026.10.28.

[8] II-4-10 баг. *Хэрэглэгчийн болон Operational гарын авлага* (II-4-10_Manual.pdf).